

Informe Técnico: Optimización y Limpieza de Código

Auditoría exhaustiva con **React Doctor** — De 134 advertencias a **0 errores y 0 advertencias**

134

ISSUES INICIALES

0

ISSUES FINALES

100%

TYPESCRIPT CLEAN

9 / 9

RUTAS EN BUILD OK

1. Rendimiento UI y GPU

63 warnings solucionados

Eliminación de transition-all indiscriminado

RENDIMIENTO

¿Por qué estaba mal? transition-all obliga al navegador a escuchar cambios y animar absolutamente todas las propiedades CSS (incluyendo padding, margin, width y height). Esto provoca recálculos de geometría (*Layout Trashing*) y caídas de cuadros por segundo (FPS) en celulares al hacer hover o clic. Solo deben animarse propiedades aceleradas por hardware GPU o variables específicas.

¿Dónde? A lo largo de app/alumnos/page.tsx, app/finanzas/page.tsx, app/rutinas/page.tsx, etc.

✗ CÓMO ESTABA

```
<Button
  className="h-11 rounded-xl bg-blue-600
  hover:bg-blue-500
  transition-all duration-300"
>
  Guardar
</Button>
```

✓ CÓMO TIENE QUE ESTAR

```
<Button
  className="h-11 rounded-xl bg-blue-600
  hover:bg-blue-500 hover:-translate-y-0.5
  transition-[color,background-color,box-
  shadow,transform]
  duration-200"
>
  Guardar
</Button>
```

2. Accesibilidad (a11y) y Formularios

26 warnings solucionados

Eliminación de autoFocus invasivo en modales

ACCESIBILIDAD

¿Por qué estaba mal? Poner autoFocus en un input dentro de un modal secuestra el foco de la pantalla de forma agresiva. Los usuarios que navegan con teclado o lectores de pantalla pierden el contexto del título y descripción del modal, saltándose información crítica.

¿Dónde? app/alumnos/page.tsx (modal de alta).

✗ CÓMO ESTABA

```
<input
  autoFocus
  value={nombre}
  placeholder="Ej. Martín Gómez"
/>
```

✓ CÓMO TIENE QUE ESTAR

```
<input
  value={nombre}
  placeholder="Ej. Martín Gómez"
/>
```

Etiquetas accesibles (aria-label e id) en controles e íconos

ACCESIBILIDAD

¿Por qué estaba mal? Un botón que solo tiene un ícono (como una X para cerrar o un tacho de basura Trash) es invisible para un lector de pantalla o control por voz, porque no tiene texto legible. Lo mismo ocurre con buscadores sin aria-label.

¿Dónde? Buscadores y botones de cierre en Finanzas, Alumnos, Rutinas y Videoteca.

✗ CÓMO ESTABA

```
<button onClick={cerrar}>
  <X className="size-5" />
</button>

<input placeholder="Buscar alumno..." />
```

✓ CÓMO TIENE QUE ESTAR

```
<button onClick={cerrar} aria-label="Cerrar
modal">
  <X className="size-5" />
</button>

<input
  id="buscar-alumnos"
  aria-label="Buscar alumno"
  placeholder="Buscar alumno..."
/>
```

Elementos no interactivos con onClick (Click Handlers en div)

ACCESIBILIDAD

¿Por qué estaba mal? Asignar eventos de clic a elementos `<div>` o `<h3>` impide que una persona navegando con la tecla Tab o Enter pueda activarlos. Un elemento cliqueable debe ser semánticamente un `<button type="button">`.

¿Dónde? `app/videoteca/page.tsx` (tarjetas de previsualización de video).

❌ CÓMO ESTABA

```
<div
  onClick={() => setVideo(v)}
  className="cursor-pointer rounded-2xl ..."
>
  <Play />
</div>
```

✅ CÓMO TIENE QUE ESTAR

```
<button
  type="button"
  aria-label={`Ver video: ${v.titulo}`}
  onClick={() => setVideo(v)}
  className="w-full text-left rounded-2xl ..."
>
  <Play />
</button>
```

3. Arquitectura y Mantenibilidad de Componentes

Modularización y Refactor

Desacople de componentes gigantes (no-giant-component)

MANTENIBILIDAD

¿Por qué estaba mal? El componente Page del Home superaba las 330 líneas de código, mezclando estadísticas del gimnasio, geolocalización, saludo al alumno y toda la lógica del formulario modal de alta. Esto incrementa la complejidad cognitiva y ralentiza los re-renders.

Solución: Se extrajo el formulario al componente `components/modal-nuevo-alumno.tsx`, reduciendo la página principal a menos de 200 líneas limpias.

¿Dónde? `app/page.tsx` y `components/modal-nuevo-alumno.tsx`.

❌ CÓMO ESTABA (TODO JUNTO EN PAGE.TSX)

```
export default function Page() {
  const [form, setForm] = useState(...)
  const [errores, setErrores] = useState(...)
  function validar() { ... }
  // +120 líneas de formulario modal embebido
  return ( ... )
}
```

✅ CÓMO TIENE QUE ESTAR (MODULARIZADO)

```
import { ModalNuevoAlumno } from
'@/components/modal-nuevo-alumno'

export default function Page() {
  const [showModal, setShowModal] =
    useState(false)
  return (
    <>
      {/* Dashboard principal */}
      <ModalNuevoAlumno
        isOpen={showModal}
        onClose={() => setShowModal(false)}
      />
    </>
  )
}
```

Cumplimiento de React Fast Refresh (only-export-components)

ARQUITECTURA

¿Por qué estaba mal? En Next.js, exportar funciones u objetos que no son componentes React (como `buttonVariants`) desde el mismo archivo del componente (`button.tsx`) rompe la recarga rápida sin pérdida de estado (Hot Module Replacement / Fast Refresh).

Solución: Se extrajo `buttonVariants` al módulo independiente `lib/button-variants.ts`.

¿Dónde? `components/ui/button.tsx` y `lib/button-variants.ts`.

❌ CÓMO ESTABA (BUTTON.TSX)

```
// button.tsx
export const buttonVariants = cva(...) // ⚠️
Rompe HMR
export function Button(...) { ... }
```

✅ CÓMO TIENE QUE ESTAR

```
// lib/button-variants.ts
export const buttonVariants = cva(...)

// components/ui/button.tsx
export { Button } // ✅ Solo exporta componentes
```

4. Concurrencia y Efectos Secundarios (Bugs)

Control de Fugas y Carreras

Extracción de fetch en useEffect con AbortController

BUGS / CONCURRENCIA

¿Por qué estaba mal? Llamar a `fetch()` directamente adentro de un `useEffect` sin señal de cancelación provoca que, si el usuario navega a otra pantalla antes de que la API responda, se intente actualizar el estado de un componente ya destruido (Memory Leak). Además, en React 19 / StrictMode las peticiones se disparan dos veces sin control de carrera.

Solución: Se extrajo la consulta a `lib/geocoding.ts` y se implementó `AbortController` con función de limpieza.

¿Dónde? `app/page.tsx` y `lib/geocoding.ts`.

❌ CÓMO ESTABA

```
useEffect(() => {
  navigator.geolocation.getCurrentPosition(async
    (pos) => {
      const res = await
        fetch(`https://api.geocoding...?
lat=${pos.coords.latitude}...`)
      const data = await res.json()
      setUbicacion(data.city) // ⚠️ Posible
memory leak
    })
}, [])
```

✅ CÓMO TIENE QUE ESTAR

```
useEffect(() => {
  const controller = new AbortController()
  navigator.geolocation.getCurrentPosition((pos)
    => {

    obtenerCiudadPorCoordenadas(pos.coords.latitude,
      pos.coords.longitude, controller.signal)
      .then((loc) => {
        if (!controller.signal.aborted)
          setUbicacion(loc)
      })
    })
  return () => controller.abort() // ✅
  Limpieza segura
}, [])
```

5. Semántica de Diálogos y Modales Nativos

Accesibilidad y Estándares Web

Uso de <dialog open> en lugar de <div role="dialog">

ESTÁNDARES HTML5

¿Por qué estaba mal? Armar un modal con un <div role="dialog"> flotante obliga al programador a escribir código manual para atrapar el foco (*Focus Trap*) y detectar la tecla Escape. Los usuarios de teclado podían "salirse" del modal y presionar botones que estaban detrás. El elemento nativo del navegador <dialog> garantiza aislamiento accesible nativo.

¿Dónde? Modales en app/planes/page.tsx, app/videoteca/page.tsx y components/modal-nuevo-alumno.tsx.

✗ CÓMO ESTABA

```
<div
  className="fixed inset-0 z-50 flex items-
center ..."
  role="dialog"
  aria-modal="true"
  aria-labelledby="modal-title"
>
  <div className="bg-white p-8 ..." > ... </div>
</div>
```

✓ CÓMO TIENE QUE ESTAR

```
<dialog
  open
  aria-labelledby="modal-title"
  className="fixed inset-0 z-50 m-0 flex h-full
max-h-none
w-full max-w-none items-center justify-center
border-none
bg-slate-950/80 p-4 backdrop-blur-sm
backdrop:bg-transparent"
>
  <div className="bg-white p-8 ..." > ... </div>
</dialog>
```

6. Seguridad en Cadena de Suministro (Supply Chain)

Hardening de Dependencias

Protección contra paquetes maliciosos (require-pnpm-hardening)

SEGURIDAD

¿Por qué estaba mal? Cuando un gestor de paquetes descarga dependencias de npm sin un período de enfriamiento (*release age*), se corre el riesgo de instalar versiones recién publicadas que contengan código malicioso (ataques de supply-chain que son dados de baja por npm a las pocas horas).

Solución: Se configuró una ventana de seguridad de 7 días (10.080 minutos) y política de no degradación de confianza.

¿Dónde? Archivo de configuración en la raíz del proyecto [pnpm-workspace.yaml].

✗ CÓMO ESTABA

```
# Archivo inexistente o sin políticas de veto
temporal
# Se aceptan paquetes publicados hace minutos
```

✓ CÓMO TIENE QUE ESTAR (PNPM-WORKSPACE.YAML)

```
packages:
  - '.'
minimumReleaseAge: 10080 # 7 días de veto
comunitario
trustPolicy: no-downgrade # Bloquea degradación
de firmas
```

7. Estabilidad de Estado Global

Hooks y Referencias

¿Por qué estaba mal? En el Store central (`lib/store.tsx`), las funciones que agregan o editan alumnos se recreaban en cada render porque dependían del array `alumnos`. Esto provocaba que cualquier componente que consumiera `useAppData()` se volviera a renderizar innecesariamente.

Solución: Se envolvieron todas las acciones en `useCallback` usando funciones de actualización funcional `setAlumnos(prev => [...prev, nuevo])`.

¿Dónde? [`lib/store.tsx`].

✗ CÓMO ESTABA

```
const agregarAlumno = (data) => {  
  const nuevo = { ...data, id:  
    Date.now().toString() }  
  setAlumnos([...alumnos, nuevo]) // ⚠ Depende  
  de 'alumnos'  
}
```

✓ CÓMO TIENE QUE ESTAR

```
const agregarAlumno = useCallback((data) => {  
  const nuevo = { ...data, id:  
    Date.now().toString() }  
  setAlumnos((prev) => [...prev, nuevo]) // ✓  
}, [])
```